

Kanan Labs: Master Strategic Architecture and Execution Roadmap (V4)

SKELETONS

Part I: The Strategic Context

Focus: Synthesizing the macro-economic market opportunity and the institutional mission.

- **1.1 The Market Reality:** Structural vs. cyclical complexity in global trade (CBAM, CSDDD, IRA).
- **1.2 The Disintermediation of Consultancies:** Addressing the MSME financial barrier to tier-one advisory retainers despite equal exposure to Global Value Chains (GVCs).
- **1.3 The 5 Core Theses the MVP Proves:** * *Drafting Note: Will explicitly define the 5 hypotheses regarding deterministic AI, operational leverage, and verifiable compliance.*

Part II: The Product Definition

Focus: Tangible definition of the proprietary assets, intake mechanisms, and core computational engines.

- **2.1 The Extensible Taxonomy (Level 1 to Level 4):** Mapping industrial sectors to specific HS codes, anchored by concrete examples (e.g., Indian EV sector, Lithium-ion accumulators / HS 850760).
- **2.2 The Firm Digital Twin:** The frictionless MSME intake structure (Revenue Bands, Certifications, Manufacturing Locations, Supply Chain Tier Level).
- **2.3 The Regulatory Risk Surface Engine:** Defining the deterministic outputs for export risk and ESG exposure (e.g., EU CBAM triggers based on Indian manufacturing state and HS code).
- **2.4 The Incentive Optimization Engine:** Processing alternative market pivots, PLI variant eligibility (e.g., Advanced Chemistry Cell PLI), and deadline tracking.

Part III: The Cloud-Native Architecture

Focus: The native GCP infrastructure pivot and the strict isolation of logic from probabilistic generation.

- **3.1 Architectural Paradigm:** The Decoupling of Deterministic Logic vs. Probabilistic LLMs.
- **3.2 Layers 1-4 (Data & Interface):** ToolJet CMS (GCP Compute Engine Virtual Machine) + Google Cloud SQL (PostgreSQL) for internal proprietary curation.
- **3.3 Layer 5 (Computational Routing):** Python/FastAPI deployed on GCP Cloud Run (Serverless, scale-to-zero compute).
- **3.4 Layer 6 (Advisory Translation):** Google Gemini 1.5 Pro via Vertex AI, utilizing strict XML tagging and the Forensic JSON Protocol for zero-hallucination output.

Part IV: No-Code Verification

Focus: Proving system determinism to institutional clients and investors without traditional unit testing

overhead.

- **4.1 The Golden Dataset (CSV) Methodology:** Empowering the founder to define edge-case scenarios and legally verified outputs purely via spreadsheet curation.
- **4.2 Agentic Test Execution:** Utilizing Claude Code CLI to autonomously build and execute `pytest` and `FastAPI TestClient` loops against the Golden Dataset.

Part V: Execution & Capital Strategy

Focus: Operational leverage, tooling economics, and milestone-driven funding.

- **5.1 The Agentic Tooling Budget:** Structuring the \$125/month operational footprint (Claude Max Plan, Lovable Pro, Google AntiGravity, free-tier Cursor).
- **5.2 The Phased Cloud Credit Acquisition:** Strategic sequencing from baseline subsidies (GCP Start Tier, AWS Founders) to institutional validation (MeitY TIDE 2.0 / SAMRIDH) to unlock Scale/AI Tiers.
- **5.3 The 6-Month Agentic Roadmap:** A strictly serialized timeline separating "Founder Domain Tasks" (ontology mapping, dataset curation) from "Claude Code CLI Tasks" (infrastructure provisioning, CI/CD, Python generation).

Part I: The Strategic Context

1. Founder & Strategic Context

[Established Facts]

Kanan Labs is founded and led by Sreenath Govindarajan, a global law graduate trained in multi-jurisdictional legal systems (spanning common and civil law across five jurisdictions) with a background in business administration. The founder's domain expertise resides at the intersection of AI governance, regulatory strategy, digital trade policy, and geopolitical risk. This background provides the specialized capacity to translate international legal elements—ranging from Global Value Chains (GVCs) and ESG Compliance norms to International Trade Law—into structured, decision-relevant intelligence.

Regulatory and geopolitical complexity in global trade is increasing structurally, not cyclically. Exporters and manufacturers must simultaneously interpret and comply with overlapping industrial policies, trade realignments, export controls, ESG mandates, subsidy regimes, and localization rules.

[Institutional Positioning]

Traditionally, the burden of navigating this overlapping complexity has been addressed through highly expensive, labor-intensive consulting firms (e.g., the Big Three), trade advisors, and regulatory counsel. Kanan Labs is designed to disintermediate this advisory layer by replacing human-capital-heavy scenario modeling with a computationally scalable, AI-native regulatory intelligence infrastructure.

The long-term vision of Kanan Labs is to function as an AI-powered consulting firm that continuously monitors global policy shifts, maps them against a firm's operational structure, identifies risk vectors, optimizes incentive capture, and produces board-level strategic intelligence on demand.

[Inferred Assessment]

Micro, Small, and Medium Enterprises (MSMEs), mid-sized exporters, and manufacturing firms integrated into GVCs possess the same structural exposure to geopolitical and regulatory risks as multinational corporations. However, they lack the financial resources to sustain recurring premium advisory retainers. A massive, highly scalable market exists for subscription-grade regulatory strategy if the cognitive labor of trade lawyers and consultants can be algorithmically automated, driving the marginal cost of advisory down to near-zero.

2. The Core Theses of the MVP

[Established Facts]

The Minimum Viable Product (MVP) does not attempt full multi-agent geopolitical simulation or autonomous capital allocation. Instead, it builds the foundational data substrate upon which those advanced layers will eventually operate.

The MVP is explicitly designed to prove five core theses to prospective institutional investors and enterprise clients:

1. **Structural Modeling:** Regulatory exposure can be structurally and mathematically modeled.
2. **Scalable Applicability:** Applicability logic can scale seamlessly across complex product classifications without requiring custom code per product.
3. **Algorithmic Optimization:** Domestic and international incentive capture can be algorithmically optimized.
4. **Deterministic Precedence:** Structured, deterministic intelligence must always precede and constrain conversational AI to prevent legal hallucination.
5. **Agentic Evolution:** This data architecture can securely evolve into a fully agentic advisory engine.

3. Vertical Focus and Jurisdictional Scope

[Institutional Positioning]

To prove computational scalability within a constrained 6-month build phase, the MVP strictly limits its sectoral focus to the **Indian Electric Vehicle (EV) components sector**, specifically targeting export-oriented manufacturing firms. This sector is selected because it is highly policy-dense, incentive-heavy, heavily exposed to export markets, subject to localization mandates, and sits at the epicenter of modern industrial policy competition.

[Established Facts]

The MVP's jurisdictional regulatory coverage is geographically constrained to high-impact trade vectors to ensure data accuracy during the manual curation phase.

- **Domestic Baseline:** India (Central Government + 2 Key States, e.g., Tamil Nadu, Maharashtra). Focus:

Production Linked Incentives (PLIs), EV schemes, and state-level capital subsidies.

- **Export Constraints (The "Shocks"):** European Union (focused on CBAM and CSDDD exposure) and the United States (focused on IRA Foreign Entity of Concern sourcing restrictions and export controls).
- **Export Opportunities (The "Pivots"):** Target markets covered by major Indian Free Trade Agreements (e.g., UAE via CEPA, Australia via ECTA, ASEAN).

4. The Product Architecture (The Core Engines)

The software product is defined by four interconnected, deterministic engines. It does not rely on probabilistic LLM scoring for risk classification; AI is utilized strictly at the end of the pipeline for narrative translation.

A. The Extensible Taxonomy Framework

[Established Facts]

The system categorizes products using a structured relational hierarchy aligned with international standards, specifically the WTO Harmonized System (HS) codes. The MVP encompasses 40-80 structured product nodes to demonstrate cross-product applicability.

- **Level 1:** Broad Sector (e.g., Automotive)
- **Level 2:** Sub-Sector (e.g., EV Components)
- **Level 3:** Functional Category (e.g., Battery Systems, Thermal Systems, Power Electronics)
- **Level 4:** HS Code Alignment (e.g., HS 8507.60 for Lithium-ion accumulators)

[Inferred Assessment]

This hierarchical structure guarantees architectural scalability. If a regulation broadly targets "Automotive" (Level 1), the database cascade automatically applies that regulatory constraint down to all corresponding Level 4 nodes. This allows Kanan Labs to expand into adjacent industries (e.g., Aerospace, Semiconductor manufacturing) purely by adding new branches to the data taxonomy, without rewriting the backend software.

B. The Firm Digital Twin (Lite)

[Institutional Positioning]

To eliminate the billable hours traditionally required for "client state assessments," Kanan Labs utilizes a frictionless, 90-second onboarding UI.

[Established Facts]

The intake module captures specific operational parameters to produce a standardized, internal JSON object (The Digital Twin). Variables captured include:

- Product classifications (mapped exactly to the Level 4 Taxonomy).
- Current and target export markets.
- Revenue bands and Tier level.
- Certifications held (ESG, Safety, ISO).
- Manufacturing states and scheme participation history.

C. The Regulatory Risk Surface Engine

[Established Facts]

Risk is calculated via pure deterministic intersection logic: Firm Digital Twin + Regulatory Graph = Applicability Matrix.

For every client profile processed, the engine generates categorical exposure outputs across the following dimensions:

- **Export Risk:** (Low/Medium/High) - e.g., Flagging US IRA sourcing exclusions.
- **Domestic Regulatory Risk:** (Low/Medium/High)
- **Certification & Standards Exposure:** Identifying missing mandatory compliance certificates for a target export market.
- **ESG & Environmental Compliance Exposure:** e.g., EU CBAM reporting requirements triggered by specific HS codes.

D. The Incentive Optimization Engine

[Institutional Positioning]

Moving beyond risk identification, the product actively structures domestic opportunity capture, transforming fragmented, hundreds-of-pages-long government gazettes into actionable financial intelligence.

[Established Facts]

By mapping the Firm Digital Twin against domestic schemes (e.g., the PLI for Advanced Chemistry Cells), the Incentive Engine deterministically:

- Determines strict legal eligibility.
- Identifies stacking conflicts (where claiming one state subsidy nullifies a central subsidy).
- Tracks filing deadlines.
- Generates customized documentation compliance checklists for the firm.

[Inferred Assessment]

By synthesizing the Risk Surface Engine with the Incentive Optimization Engine, Kanan Labs transitions from a passive "compliance monitoring tool" to an active "scenario modeling advisor." If the Risk Engine identifies that Market A is blocked by a new regulation, the system automatically calculates the optimum strategic pivot to Market B utilizing FTA concessions, while simultaneously outputting the domestic subsidies required to finance the transition.

Part II: The Product Definition

2.1 The Extensible Taxonomy (Level 1 to Level 4)

[Established Facts]

Global trade regulatory frameworks rely on rigid, hierarchical classification systems, primarily the Harmonized System (HS) nomenclature. Tariff rates, Rules of Origin (RoO), and eligibility for domestic industrial subsidies—such as India's Production Linked Incentive (PLI) schemes—are strictly tethered to these specific 6-to-8 digit numeric codes. The intersection of these codes with multiple, overlapping Free Trade Agreements (FTAs) creates a highly fragmented data environment.

[Institutional Positioning]

The foundational intellectual property of Kanan Labs is the Extensible Taxonomy—a proprietary relational database that maps parent-child industrial hierarchies directly to regulatory logic. For the MVP, this taxonomy is anchored in the Indian Electric Vehicle (EV) sector.

The mapping structure operates across four levels:

- **Level 1 (Sector):** Automotive / Mobility
- **Level 2 (Sub-Assembly):** EV Battery Systems
- **Level 3 (Component):** Lithium-ion Cells
- **Level 4 (Classification & Mapping):** HS Code 850760.

This strict taxonomy allows the system to establish definitive linkages. For instance, HS 850760 is programmatically mapped to the India-Australia Economic Cooperation and Trade Agreement (ECTA) yielding a 0% target duty, while simultaneously mapped to local Indian GST structures which differentiate between Completely Knocked Down (CKD) kits and pre-assembled battery packs.

[Inferred Assessment]

The manual, domain-expert curation of this taxonomy creates a highly defensible enterprise moat. Generic LLM tools cannot reliably synthesize this depth of intersecting legal and tariff data without hallucination. By structuring this knowledge into relational PostgreSQL tables, Kanan Labs converts static legal expertise into a highly scalable, machine-readable asset.

2.2 The Firm Digital Twin (Intake Structure)

[Established Facts]

A primary failure point for enterprise compliance software targeting Micro, Small, and Medium Enterprises (MSMEs) is "intake friction." Lengthy, highly technical onboarding questionnaires lead to high abandonment rates and incomplete data submissions, which subsequently corrupt downstream risk modeling.

[Institutional Positioning]

Kanan Labs abstracts this complexity through a frictionless, 90-second MSME intake interface. Rather than asking users to interpret trade law, the system collects universally understood business parameters to construct a "Firm Digital Twin." The structured payload captures:

- **Geographic Footprint:** Specific manufacturing states (e.g., Tamil Nadu vs. Maharashtra), which are critical for state-level incentive mapping and localized logistics constraints.
- **Financial Profile:** Annual revenue bands and export volumes.

- **Supply Chain Tier:** Positioning within the Global Value Chain (e.g., Tier 2 component manufacturer vs. final assembler).
- **Compliance Baseline:** Existing labor and environmental certifications (e.g., ISO 14001, SA8000).

This frontend data is intercepted by strict Pydantic JSON schemas. If required fields are missing or mistyped, the system halts execution, ensuring zero garbage-in-garbage-out (GIGO) errors before the data touches the computational backend.

[Inferred Assessment]

Standardizing the corporate profile into a highly structured JSON payload allows the backend engines to execute complex mathematical cross-referencing without further human intervention. The Firm Digital Twin serves as the permanent computational proxy for the client.

2.3 The Regulatory Risk Surface Engine

[Established Facts]

Market access to major Western economies is increasingly contingent on non-tariff barriers related to decarbonization and human rights. For example, an Indian manufacturer exporting aluminum or steel-intensive automotive components (e.g., suspension systems under HS 870880) to the European Union is subject to mandatory emissions reporting under the CBAM, carrying severe financial penalties for non-compliance. Similarly, the US IRA imposes strict sourcing constraints on battery minerals to qualify for consumer tax credits.

[Institutional Positioning]

The Regulatory Risk Surface Engine acts as the primary deterministic logic gate. It ingests the Firm Digital Twin and intersects the client's specific HS codes, manufacturing locations, and target markets against the Extensible Taxonomy constraints database. The engine evaluates Boolean conditionals to map risk exposure:

- *If Target Market = EU and HS Code triggers CBAM annex and Firm Tier = Component Supplier, then flag critical reporting constraint.*
- *If Target Market = US and HS Code = 850760 (Li-ion) and Origin = India, then flag IRA sourcing restriction.*

This engine does not generate text; it outputs a mathematically verified risk parameter, completely neutralizing the risk of LLM hallucination in the identification of legal liabilities.

[Inferred Assessment]

By providing immediate, deterministic visibility into geopolitical and regulatory blind spots, this engine delivers the core defensive value of a traditional tier-one supply chain consultancy. It mathematically proves to the MSME that their current export trajectory contains structural vulnerabilities.

2.4 The Incentive Optimization Engine

[Established Facts]

Global trade advisory is not solely defensive; it requires proactive commercial optimization. Governments actively deploy industrial policy to capture supply chains. India's Production Linked Incentive (PLI) for Advanced Chemistry Cells (ACC), for instance, mandates specific domestic localization requirements to unlock capital subsidies. Identifying alternative, highly profitable export routes requires triangulating domestic incentives, bilateral tariff concessions, and actual market demand.

[Institutional Positioning]

Once the Risk Surface Engine identifies a constraint, the Incentive Optimization Engine executes an algorithmic pivot to generate commercial ROI.

- **Alternative Market Calculation:** It queries the trade agreement tables for jurisdictions offering zero-tariff treatment for the specific HS code (e.g., pivoting away from the EU to the UAE under the CEPA).
- **Volume Validation:** To ensure the pivot is commercially viable, the engine cross-references the alternative market against static UN Com Trade data stored in the database, verifying historical import demand for that precise component.
- **Incentive Alignment:** It evaluates the Firm Digital Twin against domestic schemes (e.g., checking if the firm's manufacturing state and localization levels qualify them for the ACC PLI).

The engine compiles these findings into a highly structured JSON payload—containing the identified risk, the calculated alternative market, the applicable FTA, the market volume, and an array of sequential compliance steps.

[Inferred Assessment]

This engine elevates Kanan Labs from a standard compliance reporting tool into a strategic revenue optimization platform. By automatically calculating legally sound, commercially viable supply chain pivots based on empirical volume data, the platform provides actionable, boardroom-level intelligence at zero marginal cost.

Part III: The Cloud-Native Architecture

3.1 Architectural Paradigm: The Decoupling of Deterministic Logic vs. Probabilistic LLMs

[Established Facts]

Large Language Models (LLMs) are probabilistic engines that generate text by predicting the next most likely token based on the statistical distribution of their training data. While they excel at linguistic fluency and unstructured text synthesis, they possess fundamental structural limitations when tasked with executing deterministic business rules, mathematical calculations, or multi-step legal compliance logic. The AI-native development ecosystem has evolved to utilize autonomous terminal-based agents (e.g., Anthropic's Claude Code CLI) capable of reading entire codebases, configuring Docker containers, and executing local system commands to build underlying infrastructure.

[Institutional Positioning]

The Kanan Labs architecture relies on a strict, non-negotiable bifurcation between human domain expertise, deterministic algorithmic logic, and probabilistic AI synthesis. The architecture completely firewalls the computational assessment of legal and trade constraints from the LLM.

Furthermore, the engineering execution of this architecture abandons traditional manual coding. Anthropic's Claude Code Command Line Interface (CLI) is designated as the sole primary backend engineering agent. The founder acts purely as the systems architect, deploying natural language prompts to Claude Code CLI, which autonomously translates these instructions into production-ready Python code, PostgreSQL schema configurations, and `gcloud` deployment commands.

[Inferred Assessment]

By structurally isolating the LLM from the core mathematical and legal evaluation process, Kanan Labs eliminates the existential risk of hallucination (e.g., inventing non-existent tariff concessions or miscalculating Rules of Origin). This architectural firewall guarantees that the system output is mathematically, legally, and logically verifiable before any data is passed to the LLM for client presentation. Simultaneously, delegating DevOps complexity to an autonomous CLI agent transforms terminal-based cloud configuration from a prohibitive technical barrier into a standard prompting exercise, unlocking massive operational leverage for a solo founder.

3.2 Layers 1-4 (Data & Interface): ToolJet CMS (GCP Compute Engine) + Cloud SQL

[Established Facts]

A highly complex relational database is required to map the Kanan Labs Extensible Taxonomy (e.g., linking HS 850760 for Lithium-ion accumulators to multiple overlapping FTAs and domestic PLIs). While managed Backend-as-a-Service (BaaS) platforms like Supabase provide rapid deployment and built-in graphical user interfaces (GUIs), they inherently limit the ability to absorb native institutional cloud credits. ToolJet is an open-source, low-code platform that allows developers to build internal administrative panels by connecting directly to external PostgreSQL databases. Deploying ToolJet requires two distinct databases: an application database (`PG_DB`) for internal configurations and a secondary database (`TOOLJET_DB`) for metadata.

[Institutional Positioning]

The V4 architecture deprecates third-party BaaS platforms to exclusively utilize native Google Cloud Platform (GCP) primitives, specifically targeting eligibility for the Google for Startups Cloud Program.

1. **Stateful Relational Data:** The Extensible Taxonomy is housed within Google Cloud SQL, providing a fully managed, enterprise-grade PostgreSQL environment.
2. **Proprietary Data Interface (Legal CMS):** To allow the non-technical founder to visually curate dense legal doctrines without writing raw SQL, a self-hosted instance of ToolJet is deployed onto an isolated GCP Compute Engine Virtual Machine (VM).

The provisioning relies entirely on agentic orchestration:

- The founder prompts Claude Code CLI to execute `gcloud compute instances create` to provision the Linux VM and configures VPC firewall rules to restrict port access strictly to the founder's static IP address.
- Claude Code connects via SSH, installs the Docker engine, and authors the `docker-compose.yml` and `.env` files.
- Claude Code programmatically injects the Cloud SQL connection strings, generates cryptographic keys (e.g., `LOCKBOX_MASTER_KEY`), and orchestrates the dual database routing (`PG_DB` and `TOOLJET_DB`) before executing `docker-compose up -d`.

[Inferred Assessment]

This native configuration requires significantly more systems engineering than a managed BaaS solution, but the complexity is entirely absorbed by the Claude Code CLI. The resulting infrastructure grants Kanan Labs absolute data sovereignty and a highly secure, custom administrative portal for ontology mapping, while eliminating recurring per-user licensing fees and perfectly positioning the firm to absorb six-figure GCP infrastructure grants.

3.3 Layer 5 (Computational Routing): Python/FastAPI (GCP Cloud Run)

[Established Facts]

Executing deterministic trade scenarios requires a backend framework capable of high-performance routing, asynchronous data processing, and strict data validation. Python is the industry standard for mathematical orchestration. FastAPI provides native support for asynchronous programming (`async/await`), which is critical when querying large background datasets like UN Com Trade import/export volumes. Pydantic is the standard data validation library in Python, enforcing strict JSON schema adherence before logic execution. Google Cloud Run provides serverless compute, automatically scaling stateless containers to handle requests and scaling to zero during inactivity.

[Institutional Positioning]

The core proprietary asset of Kanan Labs—the Deterministic Scenario Engine—is built in Python/FastAPI and hosted on GCP Cloud Run.

- **Ingestion & Strict Validation:** When the Lovable-generated frontend submits the "Firm Digital Twin," Pydantic intercepts the JSON payload. If required parameters (e.g., Manufacturing State, specific HS Code) are missing or typed incorrectly, Pydantic immediately throws a validation error, preventing the logic engine from processing corrupted data (preventing GIGO).
- **Deterministic Execution:** The FastAPI backend securely connects to Cloud SQL using the Cloud SQL Auth Proxy managed via strictly scoped IAM service accounts. The Python engine executes hardcoded Boolean decision trees to intersect the client profile against the regulatory constraints and calculates viable algorithmic pivots (e.g., bypassing EU CBAM via UAE CEPA).
- **Agentic Deployment:** Claude Code CLI autonomously authors the Python logic, writes the necessary Dockerfile, packages the application into a container image stored in the GCP Artifact Registry, and executes the `gcloud run deploy` command.

[Inferred Assessment]

By deploying a stateless containerized application on Cloud Run, compute costs are strictly correlated with active usage, providing near-infinite margin potential per query. Furthermore, writing the engine in Python ensures it remains entirely modular; as regulatory complexity increases, the founder can seamlessly instruct Claude Code to append new Boolean arrays to the decision trees without disrupting the core ingestion and validation infrastructure.

3.4 Layer 6 (Advisory Translation): Gemini 1.5 Pro via Vertex AI

[Established Facts]

To deliver board-level strategic intelligence, the raw, highly technical JSON output generated by the Deterministic Scenario Engine must be translated into professional, lawyer-like advisory prose. Foundational LLMs with deep context windows are required for this translation. Accessing these models via native enterprise cloud APIs (such as Google Vertex AI) allows inference costs to be billed directly to the cloud provider, making them eligible for offset via startup infrastructure credits.

[Institutional Positioning]

Kanan Labs utilizes Google Gemini 1.5 Pro, accessed exclusively via the Vertex AI API, to serve as the Layer 6 translation interface. To eliminate hallucination risks, the architecture employs the "Forensic JSON Protocol" and "Structured Prompting."

1. **Payload Injection:** The FastAPI backend compiles the mathematically validated findings (the identified risk, calculated alternative market, applicable FTA, and compliance steps) into a strict JSON payload.
2. **Context Confinement:** This payload is injected into the Gemini prompt within strict XML tags (e.g., `<determinism_payload>`).
3. **Persona Constriction:** The system prompt aggressively defines Gemini's persona solely as a formatting layer. The model is explicitly instructed: *"You may only generate advice using the data provided within the `<determinism_payload>` tags. Do not add external facts, do not invent legal statutes, and do not provide advice beyond what is explicitly detailed in the payload."*

[Inferred Assessment]

Transitioning the LLM layer from third-party APIs (like OpenAI) to Vertex AI is a calculated financial maneuver to consolidate billing and maximize the utility of the Google for Startups Cloud Program. Functionally, the Forensic JSON Protocol reduces the LLM to an advanced text-formatter bound to a strict mathematical contract. This synthesis delivers the ultimate value proposition: the conversational accessibility of generative AI backed by the absolute, auditable reliability of deterministic code.

Part IV: No-Code Verification

4.1 The Golden Dataset (CSV) Methodology

[Established Facts]

A critical vulnerability in leveraging AI coding assistants for legal and financial technology is the "Looks Right, Works Wrong" paradigm. An autonomous agent can generate syntactically perfect Python code that passes basic execution tests but contains deep, catastrophic logical errors regarding how Rules of Origin (RoO) or tariff thresholds are calculated. Traditional software engineering mitigates this via exhaustive, manually written unit tests—a process highly capital- and time-intensive. Data-Driven Testing (DDT) abstracts this by separating test execution logic from the underlying test data, typically utilizing formats like Comma-Separated Values (CSV) to feed an array of inputs into a single evaluation script.

[Institutional Positioning]

The core product offering of Kanan Labs is absolute regulatory determinism. If the AI Advisory Agent hallucinates a tariff concession within a Free Trade Agreement or routes a supply chain into a non-compliant jurisdiction, enterprise trust is permanently destroyed. To mathematically prove the determinism of the Scenario Engine without requiring the non-technical founder to author Python unit tests, the architecture employs the proprietary "Golden Dataset" methodology.

The founder operates exclusively within a spreadsheet interface, translating their deep legal domain expertise into a rigid matrix. This matrix defines highly complex, multi-jurisdictional "Firm Digital Twins" (inputs) and their legally mandated, manually verified strategic outcomes (expected outputs).

The MVP Verification Matrix (Indian EV Sector Baseline):

Test Case ID	Input: HS Code	Input: Export Market	Input: Mfg State	Expected: Constraint	Expected: Pivot Market	Expected: Applicable FTA
TC-001	850760 (Li-ion)	USA	Maharashtra	IRA Restriction	Australia	ECTA
TC-002	870880 (Suspension)	EU	Tamil Nadu	CBAM Reporting	UAE	CEPA
TC-003	850132 (EV Motor)	ASEAN	Gujarat	None	ASEAN	AIFTA

TC-004	850760 (Li-ion)	EU	Karnataka	CSDDD + CBAM	UAE	CEPA
--------	-----------------	----	-----------	--------------	-----	------

This spreadsheet (`golden_dataset.csv`) serves as the immutable source of truth. The founder is solely responsible for populating these edge cases—such as explicitly defining that exporting Li-ion cells to the EU from Karnataka triggers both CSDDD and CBAM constraints, necessitating a pivot to the UAE under CEPA.

[Inferred Assessment]

This methodology fundamentally transforms Quality Assurance (QA) from a software engineering bottleneck into a pure legal data-entry exercise. By aggressively isolating the founder's intellectual labor (defining the law) from the mechanics of automated software testing (writing the code), Kanan Labs guarantees that the engine's outputs are mathematically and legally verifiable. This establishes the rigorous, auditable baseline necessary to secure enterprise clients who require definitive proof of compliance before executing multimillion-dollar supply chain pivots.

4.2 Agentic Test Execution: Pytest and Claude Code CLI

[Established Facts]

Within the Python ecosystem, the `pytest` framework is the industry standard for test automation. It utilizes the `@pytest.mark.parametrize` decorator to systematically iterate through rows of external CSV data. Furthermore, the FastAPI framework natively includes a `TestClient` module (built upon the `httpx` library). The `TestClient` allows testing scripts to route HTTP requests directly into the Asynchronous Server Gateway Interface (ASGI) application in memory. This bypasses the need to provision an actual HTTP server, eliminating network latency and allowing thousands of simulated regulatory scenarios to be processed in seconds.

[Institutional Positioning]

To operationalize the testing pipeline without manual coding, Kanan Labs utilizes Anthropic's Claude Code Command Line Interface (CLI) as the sole backend QA engineer. The execution relies on a highly specific, repeatable agentic loop:

Step 1: The Prompt Contract

The founder initiates the verification cycle by issuing a strict natural language command to the Claude Code CLI within the terminal:

"Read `golden_dataset.csv`. Construct a comprehensive `pytest` suite utilizing `FastAPI TestClient`. For every row in the CSV, map the 'Input' variables into a Pydantic-compliant JSON payload representing the Firm Digital Twin. Send a POST request to the local `/evaluate` endpoint. Assert that the engine's generated JSON payload perfectly matches the 'Expected' output columns defined in

the CSV."

Step 2: Autonomous Scaffolding

Claude Code autonomously executes the following infrastructure tasks:

- Generates the `confest.py` file to manage the test database state, ensuring the test environment securely mocks connections to the Google Cloud SQL PostgreSQL database without altering production data.
- Constructs the parametrized Python testing loop, ensuring data types from the CSV (e.g., converting string "850760" to the required integer format if dictated by the Pydantic schema) are correctly serialized.
- Executes the test suite via the terminal.

Step 3: Abductive Refactoring and Optimization

If the deterministic Python decision trees contain a logical flaw—for example, if the engine fails to flag the IRA Restriction for TC-001 because the logic gate improperly bypassed the US target market conditional—the `pytest` output will explicitly highlight `TC-001` as a failure.

The founder does not debug the code. Instead, the founder instructs Claude Code CLI: *"Test TC-001 failed. The expected constraint is 'IRA Restriction', but the engine returned 'None'. Inspect the FastAPI routing logic evaluating the US market for HS 850760, and refactor the underlying Python branching code until the assertion passes."* Claude Code repeatedly rewrites and tests the logic until a 100% pass rate is achieved.

[Inferred Assessment]

This agentic testing pipeline operationalizes the "Extreme Capital Efficiency" thesis. The founder achieves enterprise-grade, comprehensive regression testing coverage—proving absolute determinism—without writing a single line of test assertion code. This framework creates a highly scalable product structure: as new Free Trade Agreements are ratified or new PLIs are announced by the Indian government, the founder simply appends new rows to the CSV spreadsheet. The system automatically absorbs and verifies this new intelligence at zero marginal engineering cost.

Part V: Execution & Capital Strategy

5.1 The Agentic Tooling Budget

[Established Facts]

Traditionally, scaling an enterprise-grade Software-as-a-Service (SaaS) platform required raising significant pre-seed capital exclusively to fund engineering payrolls (frontend developers, database administrators, QA testers, and DevOps engineers). The 2026 AI-native development ecosystem replaces this labor cost with tiered software subscriptions. Tooling capabilities range from free basic completions to premium, unthrottled autonomous agent loops. Specifically, deploying React frontends to independent cloud hosting (Firebase) requires source code export capabilities, which are restricted to paid tiers on generators like Lovable.

[Institutional Positioning]

Kanan Labs operates as a highly leveraged "micro-team," capping its core monthly engineering and operational burn rate at exactly \$125. The toolchain is strictly optimized for the V4 cloud-native architecture:

- **Claude Code CLI (Max Plan - \$100/mo):** Serves as the primary backend and DevOps engineer. The Max plan is non-negotiable, providing the unthrottled terminal execution and massive context windows necessary to autonomously provision GCP infrastructure and execute multi-file Python refactoring.
- **Lovable (Pro Plan - \$25/mo):** Utilized for generating the "Frictionless Intake" MSME UI. The Pro plan is mandatory as it unlocks GitHub repository synchronization, a prerequisite for Firebase Continuous Integration/Continuous Deployment (CI/CD).
- **Google AntiGravity (Public Preview - \$0/mo):** Serves as the secondary visual agent, utilized specifically for automated browser orchestration to test the ToolJet deployment and for free access to Gemini 3.1 Pro.
- **Cursor (Free Tier - \$0/mo):** Relegated to a zero-cost, localized code inspector for manual syntax reviews, avoiding unnecessary Pro upgrade fees.

[Inferred Assessment]

By replacing a multi-person engineering department with a \$125/month agentic stack, Kanan Labs achieves unprecedented capital efficiency. This converts early-stage runway entirely into time for deep domain research and strategic iteration, insulating the firm from the technical debt and high burn rates that typically destroy early-stage SaaS ventures.

5.2 The Phased Cloud Credit Acquisition

[Established Facts]

Global cloud providers offer massive infrastructure subsidies to startups, but these grants are structurally tiered. The highest tranches—such as the Google for Startups Cloud Program "AI Tier" (\$350,000) and the AWS Activate "Portfolio Package" (\$100,000)—strictly require a verified Organization ID from a recognized venture capital firm, incubator, or accelerator. Self-funded or bootstrapped startups only qualify for entry-level tiers (e.g., \$1,000 to \$2,000). The Indian government, via the Ministry of Electronics and Information Technology (MeitY), operates localized deep tech schemes (TIDE 2.0, SAMRIDH) that provide both non-dilutive capital and institutional validation.

[Institutional Positioning]

Kanan Labs will execute a strictly phased, milestone-driven capital acquisition strategy to absorb cloud costs without relying on immediate external equity financing:

1. **Phase 1: Baseline Subsidization (Month 1):** The founder generates a functional landing page via Lovable to satisfy the "active domain" prerequisite. The firm immediately secures the Google Cloud Start Tier (\$2,000) and AWS Activate Founders Tier (\$1,000). This \$3,000 buffer is mathematically sufficient to absorb the minimal footprint of Cloud Run, Cloud SQL, and Vertex AI inference during the six-month MVP build.
2. **Phase 2: Accelerator Incubation (Months 3-4):** Upon demonstrating the verified determinism of

the Scenario Engine via the "Golden Dataset" (proving the product thesis), the founder applies to MeitY-affiliated incubators (e.g., IIT Madras Research Park).

3. **Phase 3: Scale Tranche Unlocking (Month 6):** Acceptance into the incubator serves dual purposes: it provides immediate non-dilutive grant funding (e.g., up to 7 lakh via TIDE 2.0) and supplies the certified Organization ID required to legally unlock the \$350,000 GCP AI Tier, subsidizing long-term enterprise scale.

[Inferred Assessment]

Attempting to secure institutional cloud credits prematurely without an incubator affiliation or verifiable equity funding will result in automatic rejection. The phased approach guarantees uninterrupted infrastructure scaling. By utilizing the mathematically proven MVP to gain entry into Indian state-backed incubators, Kanan Labs systematically builds the institutional credibility required to unlock multi-year, six-figure cloud runways.

24-Week Roadmap

Executing a multi-container cloud architecture alongside a deeply researched legal database within six months requires ruthless operational serialization. For a solo founder utilizing agentic coding tools, allowing technical tasks to overlap with domain-research tasks leads to context switching, prompt degradation, and architectural collapse.

The roadmap is enforced as a weekly sprint matrix. The founder's role is strictly confined to two activities: **Domain Architecting** (defining what the system must know) and **Agent Orchestration** (commanding the CLI to build it). Claude Code CLI handles 100% of the syntax execution.

This matrix acts as the CTO's operational playbook. It transforms an overwhelming technical build into a predictable, mechanical sequence of inputs and outputs, ensuring the MVP hits the Month 6 funding gate without incurring technical debt.

Month 1: Infrastructure Scaffolding & Foundations

Week	Phase Objective	Founder Tasks (Domain & Strategy)	Claude CLI / Agent Tasks	Deliverable / Completion Metric	Notes
W1	Subsidy Baseline	Generate basic landing page via Lovable. Purchase domain. Apply for GCP Start Tier & AWS Founders credits.	N/A (UI generation done via Lovable visual interface).	Active domain; \$3,000 in combined cloud credits secured.	

W2	GCP Init	Draft the Level 1-4 hierarchy mapping for the Indian EV sector on paper (Sector -> Sub-Assembly -> Component -> HS Code).	Execute gcloud init. Enable Compute, SQL, and Vertex AI APIs. Configure IAM service accounts with strict least-privilege.	Secure GCP environment with authorized service accounts.	
W3	Cloud SQL	Define exact JSON parameters required for the Firm Digital Twin (Revenue, Mfg State, Tier Level).	Provision Google Cloud SQL (PostgreSQL) instance. Scaffold foundational Pydantic models in Python based on Founder's JSON specs.	Running database; Base Python codebase (main.py, models.py) initialized.	
W4	Schema Setup	Finalize the Extensible Taxonomy table relationships (e.g., how an HS Code links to a Target Market).	Autonomously write and execute SQL migration scripts to create empty relational tables based on the founder's taxonomy design.	Validated PostgreSQL schema matching the taxonomy design.	

Month 2: CMS Deployment & Regulatory Graph Curation

Week	Phase Objective	Founder Tasks (Domain & Strategy)	Claude CLI / Agent Tasks	Deliverable / Completion Metric	Notes
W5	VM Provisioning	Review ToolJet documentation to understand its UI builder capabilities.	Provision GCP Compute Engine VM. Install Docker and Docker Compose via SSH. Configure VPC firewall restricting access to	Secure Linux VM running Docker, accessible only by Founder.	

			Founder's IP.		
W6	ToolJet Deploy	Generate cryptographic master keys (using OpenSSL locally) and provide them to Claude.	Write <code>docker-compose.yml</code> and <code>.env</code> . Map PG_DB and TOOLJET_DB connections to Cloud SQL. Deploy ToolJet container.	Live ToolJet login screen accessible at the VM's IP address.	
W7	CMS Interface	Log into ToolJet. Drag and drop UI components to build the custom data-entry forms mapping to the Cloud SQL tables.	Use Google AntiGravity to visually test the ToolJet UI deployment and confirm database read/write permissions are active.	Functional internal CMS ready for data entry.	
W8	Data Injection	Core Intellectual Labor: Manually input PLI constraints, FTA Rules of Origin, and CBAM triggers for HS 850760 into ToolJet.	N/A (System administration mode; Claude stands by).	Extensible Taxonomy populated with Indian EV baseline data.	

Month 3: Market Data Ingestion & Logic Scaffolding

Week	Phase Objective	Founder Tasks (Domain & Strategy)	Claude CLI / Agent Tasks	Deliverable / Completion Metric	Notes
W9	Volume Data	Download, clean, and format historical UN Com Trade volume datasets (CSVs) for	Write asynchronous Python script to parse the CSVs and perform bulk-inserts into the Cloud SQL volume	Market volume data successfully ingested into	

		target EV components.	tables.	database.	
W10	Logic Mapping	Map the Boolean decision pathways on paper (e.g., <i>If HS 850760 and EU, Then Check CBAM, Then Pivot to UAE</i>).	Build FastAPI routing endpoints (e.g., <code>/evaluate</code>). Configure Cloud SQL Auth Proxy for secure local-to-cloud connection.	FastAPI server running locally, successfully connected to Cloud SQL.	
W11	Engine Scaffolding	Refine decision pathways, defining exact sequential compliance steps for each pivot.	Translate Founder's mapped logic into deterministic Python <code>if/else</code> decision trees and utility functions.	Python logic engine drafted (untested).	
W12	Integration	Map Pydantic intake payload strictly to the newly drafted Python decision trees.	Wire the FastAPI endpoints to the Python decision trees, ensuring the engine can accept a JSON payload and return a JSON result.	End-to-end local routing completed.	

Month 4: No-Code Verification (The Golden Dataset)

Week	Phase Objective	Founder Tasks (Domain & Strategy)	Claude CLI / Agent Tasks	Deliverable / Completion Metric	Notes
W13	Data Creation	Build <code>golden_dataset.csv</code> in Excel. Define 20-30 complex edge cases (TC-001 to TC-030) and legally verify expected outputs.	N/A	Completed Golden Dataset CSV matrix.	

W14	Test Framework	Prompt Claude with the exact testing instructions (read CSV, map to Pydantic, assert output).	Construct <code>pytest</code> framework, <code>TestClient</code> , and <code>conftest.py</code> . Write the <code>@pytest.mark.parametrize</code> loop to iterate through the CSV.	Executable automated test suite.	
W15	Execution	Review the <code>pytest</code> failure logs. Identify <i>why</i> the logic failed from a legal perspective.	Run the test suite. Log all assertion failures where the engine's JSON output does not match the CSV's expected output.	Comprehensive failure log report.	
W16	Optimization	Direct Claude's refactoring efforts: <i>"TC-004 failed because you missed the CKD vs. Pre-assembled GST split. Fix the conditional."</i>	Autonomously refactor and rewrite the Python decision trees based on Founder instructions until tests pass.	Major Milestone: 100% Pytest pass rate on the Golden Dataset.	

Month 5: LLM Translation & Guardrail Integration

Week	Phase Objective	Founder Tasks (Domain & Strategy)	Claude CLI / Agent Tasks	Deliverable / Completion Metric	Notes
W17	Prompt Eng	Draft the Forensic JSON Protocol system prompts. Define the XML tagging structure	Scaffold the Python wrapper functions required to structure the Gemini API payload using Google's	Finalized system prompt architecture.	

		(<determinism_payload>).	official SDK.		
W18	Vertex AI Init	Authorize GCP billing for Vertex AI using the Start Tier credits obtained in Week 1.	Integrate Gemini 1.5 Pro API into the FastAPI backend. Connect the deterministic JSON output to the LLM prompt context window.	LLM successfully translating test JSON payloads into advisory prose.	
W19	Red-Teaming	Conduct adversarial testing. Actively attempt to "jailbreak" the LLM to provide unverified legal advice.	Monitor API logs. Adjust API parameters (Temperature = 0.0) to ensure absolute rigidity in LLM output.	High-confidence, zero-hallucination output confirmed.	
W20	Cloud Run Deploy	Final review of the backend architecture.	Write Dockerfile, build image via GCP Artifact Registry, and execute gcloud run deploy.	Backend deployed serverless; live, secure API endpoint obtained.	

Month 6: Frontend Deployment & Scale Acquisition

Week	Phase Objective	Founder Tasks (Domain & Strategy)	Claude CLI / Agent Tasks	Deliverable / Completion Metric	Notes
W21	UI Finalization	Return to Lovable. Visually generate the final "Frictionless Intake"	N/A (Operated purely via	Complete React/TypeScript	

		forms and the Advisory Dashboard UI.	Lovable).	frontend generated.	
W22	CI/CD Pipeline	Export Lovable codebase to GitHub.	Write GitHub Actions .yaml file. Configure automated build/test/deploy pipeline to push the code to GCP Firebase Hosting.	Automated deployment pipeline established.	
W23	End-to-End QA	Run full simulations using pilot MSME profiles from frontend to backend. Identify any UI/UX or data routing snags.	Configure Cross-Origin Resource Sharing (CORS) between Firebase frontend and Cloud Run backend. Debug integration issues.	Major Milestone: Fully functional MVP live on custom domain.	
W24	Scale Tranche	Compile tech documentation and the verified Pytest logs. Submit applications to MeitY incubators (TIDE 2.0 / SAMRIDH).	Execute final security review. Document the architecture topology for institutional due diligence.	Accelerator application submitted; Ready to unlock Scale Tier funding.	